

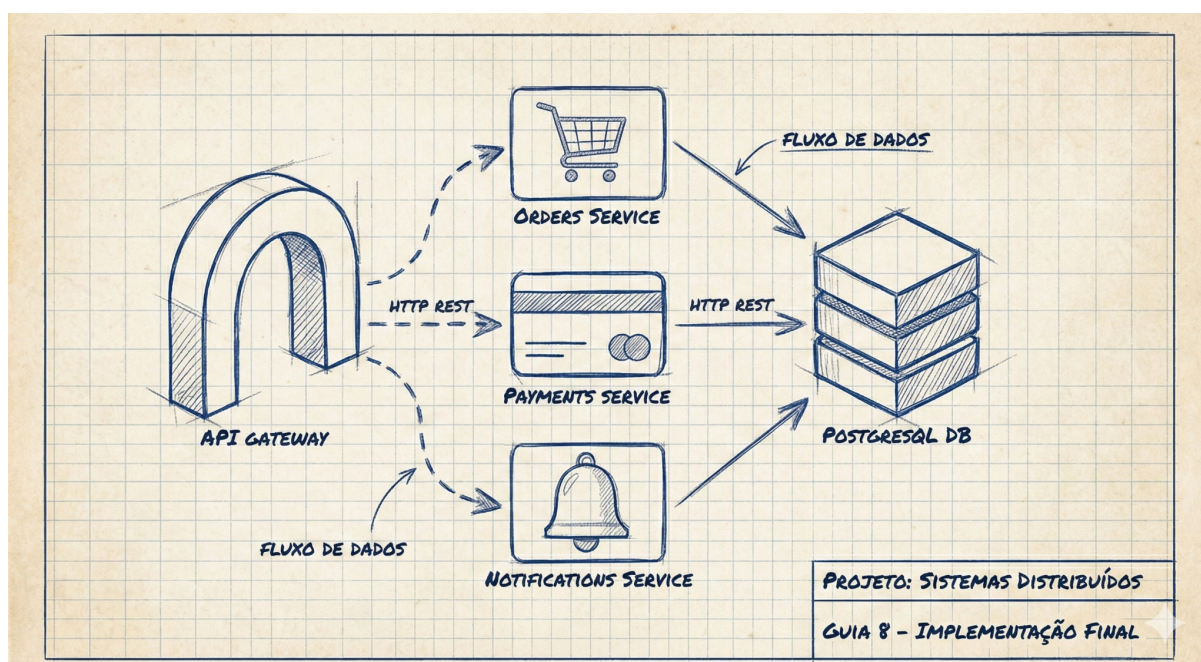
Sistemas Distribuídos

Trabalho Prático 08 - Mini Projeto

Redes e Sistemas Informáticos

Trabalho realizado por:

- João Almeida - 127140
- Júlio Santos - 127878



Introdução e Objetivos

O presente projeto tem como objetivo o desenvolvimento de um sistema distribuído para a monitorização e gestão de pedidos de compra, baseado numa arquitetura de microsserviços containerizados com Docker. Os diferentes serviços comunicam entre si através de APIs HTTP, sendo o acesso externo centralizado por um API Gateway.

O sistema foi concebido com o objetivo de cumprir os seguintes requisitos fundamentais:

- Implementação de serviços de negócio independentes (Orders, Payments e Notifications);
- Centralização do acesso aos serviços através de um API Gateway, responsável pelo encaminhamento e monitorização dos pedidos;
- Exposição de métricas de desempenho e utilização através do Prometheus;
- Visualização e análise das métricas através do Grafana;
- Centralização e monitorização de logs utilizando Loki e Promtail, permitindo uma análise unificada do comportamento dos serviços;
- Orquestração e isolamento dos serviços recorrendo a Docker Compose, garantindo portabilidade e facilidade de execução.

Índice

Introdução e Objetivos	2
Índice	3
Arquitetura da Solução	5
Componentes do Sistema	5
Observabilidade e Monitorização	6
2.1. Definição da Infraestrutura (Docker Compose)	7
API Gateway	7
Serviços de Negócio	7
Observabilidade e Monitorização	9
Rede e Comunicação	9
Considerações Técnicas	9
Implementação da Camada de Dados	10
3.1. Modelo de Dados	10
3.2. Tabela de Pedidos (Orders)	11
3.3. Tabela de Pagamentos (Payments)	12
3.4. Tabela de Notificações (Notifications)	13
3.5. Considerações Arquiteturais	14
Justificação das Decisões	14
4. Desenvolvimento dos Serviços	15
4.1. API Gateway	16
Funcionalidades principais:	16
Métricas implementadas:	16
4.2. Orders Service	17
Endpoints disponíveis:	17
Funcionamento:	17
Métricas:	17
4.3. Payments Service	18
Endpoints disponíveis:	18
Funcionamento:	18
Métricas:	18
4.4. Notifications Service	19
Endpoints disponíveis:	19
Funcionamento:	19
Métricas:	19
4.5. Logs Service	20
Endpoints disponíveis:	20
Métricas:	20
4.6. Métricas Service	21
Endpoints disponíveis:	21
Métricas:	21
4.7. Considerações Finais da Implementação	22
5. Observabilidade e Monitorização	23

5.1. Métricas (Prometheus e Grafana)	24
Instrumentação dos Serviços	24
Configuração e Validação do Prometheus	25
Visualização no Grafana	26
5.2. Logs Centralizados (Loki e Promtail)	27
Recolha de Logs (Promtail)	27
Armazenamento e Consulta (Loki)	27
Visualização no Grafana	27
5.3. Considerações sobre Observabilidade	28
6. Interface e API Gateway	29
6.1. API Gateway	29
Implementação do API Gateway	29
6.2. Interface de Consumo da API	32
6.3. Considerações Finais sobre o Gateway	33
Decisões Técnicas e Limitações do Projeto	34
Tentativa de Implementação do API Gateway com NGINX	34
7. Conclusão	36
Desacoplamento de Serviços	36
Persistência e Integridade de Dados	36
Observabilidade Completa (Prometheus, Grafana, Loki e Promtail)	37
Centralização de Acesso através de API Gateway	37
Guia Prático de Demonstração do Sistema	38
1. Arranque do sistema	38
2. Verificação de estado (Health Check)	38
3. Criação de uma order (fluxo automático)	39
4. Criação de múltiplas orders com valores diferentes	39
5. Listar orders	40
6. Listar payments	40
7. Listar notifications	40
8. Acesso direto à base de dados PostgreSQL	41
9. Geração de erros para observabilidade	43
10. Prometheus	43
11. Grafana	44
12. Logs (Loki e Promtail)	44
13. Encerramento do sistema	44

Arquitetura da Solução

A orquestração de todos os serviços é definida no ficheiro `docker-compose.yml`, responsável por criar e iniciar todos os contentores necessários ao funcionamento do sistema. Os serviços são ligados através de uma rede virtual Docker, permitindo a comunicação interna entre contentores utilizando nomes DNS dos serviços (por exemplo, **`http://orders:5001`**).

A arquitetura segue o modelo de microsserviços, onde cada componente possui uma responsabilidade bem definida e comunica através de APIs HTTP REST. O acesso externo ao sistema é centralizado através de um API Gateway, que encaminha os pedidos para os serviços apropriados e recolhe métricas de utilização.

O sistema implementa um fluxo automático que garante o encadeamento de processos sem intervenção manual. O fluxo inicia-se quando o API Gateway recebe um pedido de criação de uma order (POST `/orders`). O serviço Orders persiste o pedido na base de dados PostgreSQL e, após a persistência, realiza uma chamada síncrona ao serviço Payments para criar o pagamento associado. Por fim, é realizada uma chamada ao serviço Notifications para gerar um registo informativo. Todo este processo é transparente para o utilizador.

Componentes do Sistema

- **API Gateway (FastAPI)**

Atua como ponto único de entrada para o sistema. É responsável por:

- Encaminhar pedidos HTTP para os serviços internos;
- Agregar respostas;
Expor métricas de desempenho e utilização;
- Fornecer um endpoint de verificação de estado (health check).

- **Orders Service**

Responsável pela criação e listagem de pedidos de compra. Este serviço expõe endpoints REST e recolhe métricas relativas ao número de pedidos e tempos de resposta.

- **Payments Service**

Simula o processamento de pagamentos associados aos pedidos. O serviço responde a pedidos HTTP e expõe métricas relacionadas com o volume e latência dos pagamentos.

- **Notifications Service**

Responsável pela simulação do envio de notificações aos utilizadores após eventos relevantes, como a confirmação de pagamentos.

- **Logs Service**

Centraliza e disponibiliza logs gerados pelos diferentes serviços, permitindo a sua análise através da stack de observabilidade.

- **Métricas Service**

Serviço dedicado à exposição de métricas simuladas e globais para validação da stack de observabilidade e exposição de métricas globais do sistema, funcionando como complemento à monitorização dos restantes microserviços.

Observabilidade e Monitorização

A solução integra um sistema completo de observabilidade, composto por:

- **Prometheus**, responsável pela recolha periódica de métricas expostas por todos os serviços;
- **Grafana**, utilizado para a visualização das métricas através de dashboards personalizados;
- **Loki**, para centralização e armazenamento de logs;
- **Promtail**, responsável pela recolha e envio dos logs dos contentores para o Loki.

Esta infraestrutura permite monitorizar, em tempo real, o desempenho do sistema, incluindo número de pedidos, taxas de erro e latências.

Nota sobre NGINX: Durante o desenvolvimento do projeto foi considerada a utilização de NGINX como API Gateway e Reverse Proxy, com o objetivo de ocultar os serviços internos e melhorar a gestão do tráfego. No entanto, devido à complexidade adicional e às limitações de tempo, esta abordagem foi abandonada, tendo sido optado por um API Gateway implementado diretamente em FastAPI, solução que se revelou mais simples e adequada aos objetivos do projeto.

2.1. Definição da Infraestrutura (Docker Compose)

A infraestrutura do sistema é definida através do ficheiro **docker-compose.yml**, responsável por orquestrar todos os serviços necessários ao funcionamento da aplicação. Cada componente é executado num contentor Docker independente, promovendo isolamento, escalabilidade e facilidade de manutenção. Este ficheiro foi estruturado para permitir o arranque de todo o sistema com um único comando. As configurações incluem:

- Definição explícita de todos os serviços de negócio e de observabilidade (Prometheus, Grafana, Loki, Promtail).
- Criação de uma rede interna partilhada para comunicação entre serviços.
- Configuração de dependências (`depends_on`) para garantir a ordem correta de arranque dos contentores.

Os serviços definidos no Docker Compose estão organizados da seguinte forma:

API Gateway

- **api-gateway**

Implementado em FastAPI, atua como ponto único de entrada do sistema. Recebe todos os pedidos externos e encaminha-os para os respectivos microserviços internos. Este serviço também é responsável pela recolha e exposição de métricas globais.

Serviços de Negócio

- **orders**
Serviço responsável pela criação e listagem de pedidos de compra.
- **payments**
Serviço responsável pela simulação do processamento de pagamentos.
- **notifications**
Serviço responsável pela geração de notificações associadas aos pedidos.
- **metrics**
Serviço dedicado à agregação e exposição de métricas globais do sistema.

Todos estes serviços comunicam entre si através de **APIs HTTP REST**, utilizando os nomes dos serviços Docker como endereços internos.

Observabilidade e Monitorização

- **prometheus**
Recolhe métricas expostas por todos os serviços.
- **grafana**
Plataforma de visualização utilizada para a criação de dashboards de monitorização.
- **loki**
Sistema de agregação e armazenamento de logs.
- **promtail**
Responsável pela recolha dos logs dos contentores e envio para o Loki.

Rede e Comunicação

Todos os serviços são ligados através de uma rede Docker dedicada, permitindo a comunicação interna segura e isolada. Os endpoints de métricas (`/metrics`) são utilizados pelo Prometheus para recolha periódica de dados.

Considerações Técnicas

Inicialmente foi considerada a utilização de NGINX como Reverse Proxy e API Gateway, bem como a introdução de comunicação assíncrona via RabbitMQ. No entanto, devido à complexidade adicional e ao tempo disponível para o projeto, optou-se por uma arquitetura baseada exclusivamente em comunicação síncrona via HTTP e por um API Gateway implementado diretamente em FastAPI.

Implementação da Camada de Dados

A camada de dados do sistema foi concebida para suportar o ciclo completo de processamento de pedidos de compra, desde a sua criação até à confirmação do pagamento e emissão de notificações. A arquitetura segue um modelo lógico de separação de responsabilidades, onde cada serviço de negócio é responsável pela gestão do seu próprio conjunto de dados.

Embora o sistema seja implementado sob uma arquitetura de microsserviços, a modelação dos dados segue princípios relacionais clássicos, garantindo consistência, rastreabilidade e integridade da informação.

A persistência em base de dados relacional (PostgreSQL) garante que os dados não são perdidos entre reinícios e permite a verificação direta da consistência do fluxo automático, especificamente através da chave estrangeira `order_id` na tabela `payments`.

3.1. Modelo de Dados

O modelo de dados do sistema é composto por três entidades principais:

- **Orders** – representa os pedidos de compra efetuados
- **Payments** – representa o processamento financeiro associado a um pedido
- **Notifications** – representa eventos e notificações geradas ao longo do fluxo do pedido

Cada entidade corresponde logicamente a um serviço de negócio distinto.

3.2. Tabela de Pedidos (Orders)

A tabela "orders" armazena a informação base de cada pedido de compra criado no sistema. Cada pedido possui um estado associado que permite acompanhar a sua evolução ao longo do processo.

```
CREATE TABLE orders (  
  id SERIAL PRIMARY KEY,  
  product VARCHAR(128) NOT NULL,  
  amount NUMERIC(10,2) NOT NULL,  
  status VARCHAR(32) NOT NULL DEFAULT 'created',  
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

Descrição dos campos:

- **id**: identificador único do pedido
- **product**: descrição do produto adquirido
- **amount**: valor monetário do pedido
- **status**: estado atual do pedido (ex.: created, paid, cancelled)
- **created_at**: data de criação do pedido
- **updated_at**: data da última atualização

Este modelo permite rastrear a evolução do pedido ao longo do tempo.

3.3. Tabela de Pagamentos (Payments)

A tabela "payments" registra o processamento financeiro associado a um pedido. Cada pagamento está obrigatoriamente associado a um pedido existente.

```
CREATE TABLE payments (  
  id SERIAL PRIMARY KEY,  
  order_id INTEGER NOT NULL,  
  amount NUMERIC(10,2) NOT NULL,  
  status VARCHAR(32) NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  CONSTRAINT fk_order  
    FOREIGN KEY (order_id)  
    REFERENCES orders(id)  
);
```

Descrição dos campos:

- **id**: identificador único do pagamento
- **order_id**: referência ao pedido associado
- **amount**: valor processado
- **status**: estado do pagamento (ex.: pending, completed, failed)
- **created_at**: data de processamento

Esta estrutura garante a ligação direta entre pedidos e pagamentos.

3.4. Tabela de Notificações (Notifications)

A tabela de “notificações” mantém o histórico de eventos e mensagens geradas pelo sistema, permitindo rastrear notificações associadas a pedidos e pagamentos.

```
CREATE TABLE notifications (  
  id SERIAL PRIMARY KEY,  
  order_id INTEGER NOT NULL,  
  payment_id INTEGER,  
  type VARCHAR(64) NOT NULL,  
  message TEXT NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  CONSTRAINT fk_notification_order  
    FOREIGN KEY (order_id)  
    REFERENCES orders(id),  
  CONSTRAINT fk_notification_payment  
    FOREIGN KEY (payment_id)  
    REFERENCES payments(id)  
);
```

Descrição dos campos:

- **id**: identificador único da notificação
- **order_id**: pedido associado à notificação
- **payment_id**: pagamento associado (quando aplicável)
- **type**: tipo de evento (ex.: order_created, payment_completed)
- **message**: mensagem descritiva
- **created_at**: data da notificação

3.5. Considerações Arquiteturais

Cada serviço de negócio é responsável pela gestão lógica dos seus dados, respeitando o princípio de desacoplamento entre microserviços. A comunicação entre serviços ocorre através de chamadas HTTP e troca de eventos, evitando dependências diretas ao nível da base de dados.

Esta abordagem facilita:

- Escalabilidade independente dos serviços
- Manutenção e evolução do sistema
- Integração com sistemas de observabilidade (métricas e logs)

Justificação das Decisões

A modelação relacional adotada fornece uma base sólida para o sistema, mesmo quando os dados são simulados ou geridos temporariamente durante a execução. Esta decisão permite que a arquitetura seja facilmente estendida para ambientes de produção, bastando integrar a persistência física sem alterações significativas à lógica dos serviços.

4. Desenvolvimento dos Serviços

O sistema foi desenvolvido segundo uma arquitetura de microsserviços, onde cada componente é responsável por uma função de negócio específica. Os serviços comunicam-se entre si através de chamadas HTTP REST, sendo a **API Gateway** o ponto único de entrada para o sistema.

Todos os serviços expõem endpoints de **health check** e **/metrics**, permitindo a monitorização contínua através do Prometheus e visualização no Grafana.

4.1. API Gateway

O **API Gateway** foi implementado utilizando **FastAPI** e atua como camada de abstração entre o cliente e os serviços internos. Este componente é responsável por encaminhar os pedidos HTTP para os serviços apropriados e por agregar métricas globais do sistema.

Funcionalidades principais:

- Encaminhamento de pedidos para os serviços:
 - Orders
 - Payments
 - Notifications
 - Logs
 - Métricas
- Endpoint de saúde agregado (GET /api/health)
- Instrumentação completa com Prometheus

Métricas implementadas:

- Total de requests (api_gateway_requests_total)
- Latência dos pedidos (api_gateway_request_duration_seconds)
- Requests por segundo
- Erros por status HTTP

Estas métricas permitem analisar o comportamento global do sistema e identificar gargalos de performance.

4.2. Orders Service

O **Orders Service** é responsável pela criação e listagem de pedidos de compra. Este serviço foi implementado com Flask e persiste os dados na base de dados PostgreSQL, garantindo que a informação não se perde entre reinícios.

Endpoints disponíveis:

- GET /orders – devolve a lista de pedidos armazenados na base de dados;
- POST /orders – cria um novo pedido e inicia o fluxo automático;
- GET /health – estado do serviço;
- GET /metrics – métricas Prometheus.

Funcionamento:

Quando um pedido é criado (POST), o serviço persiste o mesmo na tabela orders e comunica imediatamente com os serviços Payments e Notifications para dar continuidade ao fluxo de negócio.

Métricas:

- Total de pedidos (orders_requests_total)
- Latência dos pedidos (orders_request_duration_seconds)
- Requests por segundo

4.3. Payments Service

O **Payments Service** gere o processamento de pagamentos. Ao contrário da versão inicial simulada, este serviço cria registos persistentes na tabela payments do **PostgreSQL**, associando-os às respetivas encomendas.

Endpoints disponíveis:

- GET /payments – lista os pagamentos registados na base de dados;
- POST /payments – cria um novo pagamento (chamada interna).
- GET /health
- GET /metrics

Funcionamento:

O serviço recebe o ID da encomenda e o montante, criando um registo com chave estrangeira para a tabela orders, assegurando a integridade referencial dos dados.

Métricas:

- Total de pagamentos processados (payments_requests_total)
- Latência de processamento (payments_request_duration_seconds)
- Requests por segundo

4.4. Notifications Service

O **Notifications Service** representa a camada final de comunicação com o utilizador. Este serviço é responsável por registar eventos do sistema. As notificações geradas durante o fluxo automático são persistidas na base de dados para consulta futura.

Endpoints disponíveis:

- GET /notifications – lista o histórico de notificações;
- POST /notifications – cria uma nova notificação.
- GET /health
- GET /metrics

Funcionamento:

As notificações são geradas automaticamente após a criação de uma encomenda e pagamento, contendo uma mensagem informativa e um timestamp de criação.

Métricas:

- Total de notificações (notifications_requests_total)
- Latência (notifications_request_duration_seconds)
- Requests por segundo

4.5. Logs Service

O **Logs Service** permite simular a consulta de logs de diferentes serviços do sistema. Este serviço complementa a observabilidade ao nível funcional, enquanto o Loki centraliza os logs reais dos contentores.

Endpoints disponíveis:

- GET /logs
- GET /health
- GET /metrics

Métricas:

- Total de pedidos ao serviço de logs (logs_requests_total)
- Latência dos pedidos

4.6. Métricas Service

O **Métricas Service** foi desenvolvido com o objetivo de gerar métricas agregadas e valores simulados, permitindo validar o funcionamento do Prometheus e dos dashboards no Grafana.

Endpoints disponíveis:

- GET /summary
- GET /health
- GET /metrics

Métricas:

- Total de requests (`metricas_requests_total`)
- Histograma de latência (`metricas_request_duration_seconds`)
- Requests por segundo

4.7. Considerações Finais da Implementação

Apesar de os dados serem simulados e mantidos em memória, a estrutura do sistema reflete uma arquitetura de microsserviços realista, permitindo:

- Testar comunicação entre serviços
- Medir desempenho e latência
- Validar ferramentas de observabilidade
- Visualizar métricas em tempo real no Grafana

Esta abordagem permite focar o projeto nos conceitos fundamentais de **Sistemas Distribuídos**, nomeadamente modularidade, monitorização e escalabilidade.

5. Observabilidade e Monitorização

A observabilidade foi um dos pilares centrais do projeto, permitindo analisar o comportamento do sistema distribuído em tempo real. A solução integra métricas, logs centralizados e dashboards, recorrendo a ferramentas amplamente utilizadas na indústria: **Prometheus, Grafana, Loki e Promtail**.

Ao contrário de abordagens baseadas em dados fictícios, todas as métricas e logs apresentados refletem **execuções reais dos serviços**, geradas a partir de pedidos HTTP efetuados durante o funcionamento do sistema.

5.1. Métricas (Prometheus e Grafana)

A recolha de métricas é assegurada pelo **Prometheus**, configurado para realizar scraping periódico dos endpoints `/metrics` expostos por todos os microsserviços, incluindo o **API Gateway**.

Instrumentação dos Serviços

Cada serviço foi instrumentado utilizando a biblioteca `prometheus_client`, expondo métricas reais de execução, nomeadamente:

- **Contadores (Counter)** para o número total de pedidos processados;
- **Histogramas (Histogram)** para medição da latência dos pedidos HTTP.

Entre as métricas implementadas destacam-se:

- `api_gateway_requests_total`
- `orders_requests_total`
- `payments_requests_total`
- `notifications_requests_total`
- `metrics_requests_total`
- Histogramas de latência do tipo `*_request_duration_seconds`

Todos os serviços disponibilizam o endpoint:

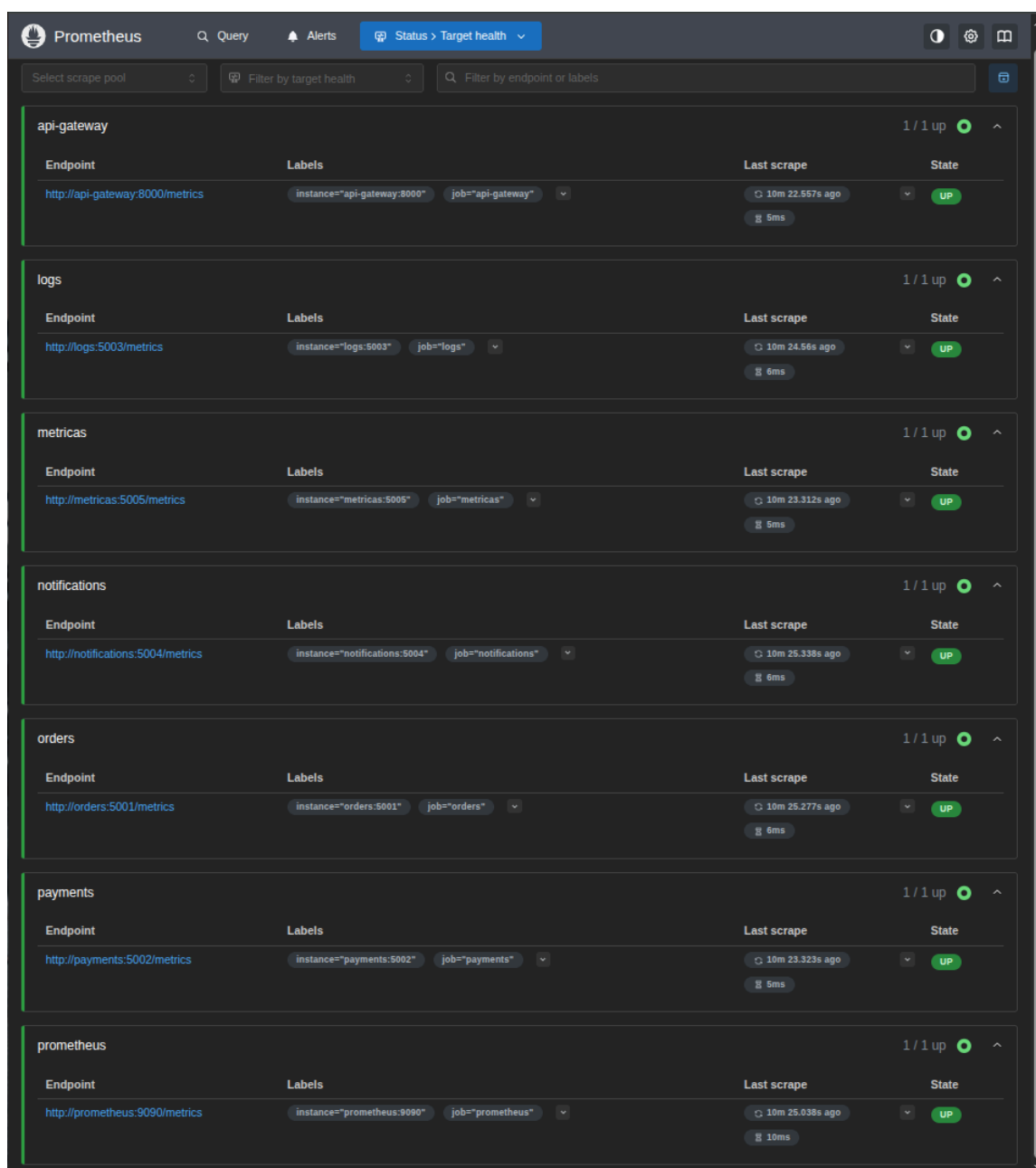
GET `/metrics`

permitindo ao Prometheus recolher automaticamente as métricas através da rede interna do Docker.

Configuração e Validação do Prometheus

O Prometheus encontra-se acessível na porta **9090** e foi configurado através do ficheiro `prometheus.yml`, onde são definidos **jobs individuais para cada microsserviço**.

A correta configuração e funcionamento da recolha de métricas foi validada através da interface web do Prometheus, verificando-se que **todos os targets se encontram no estado UP**, conforme ilustrado na **Figura**, não existindo falhas de scraping nem erros de configuração.



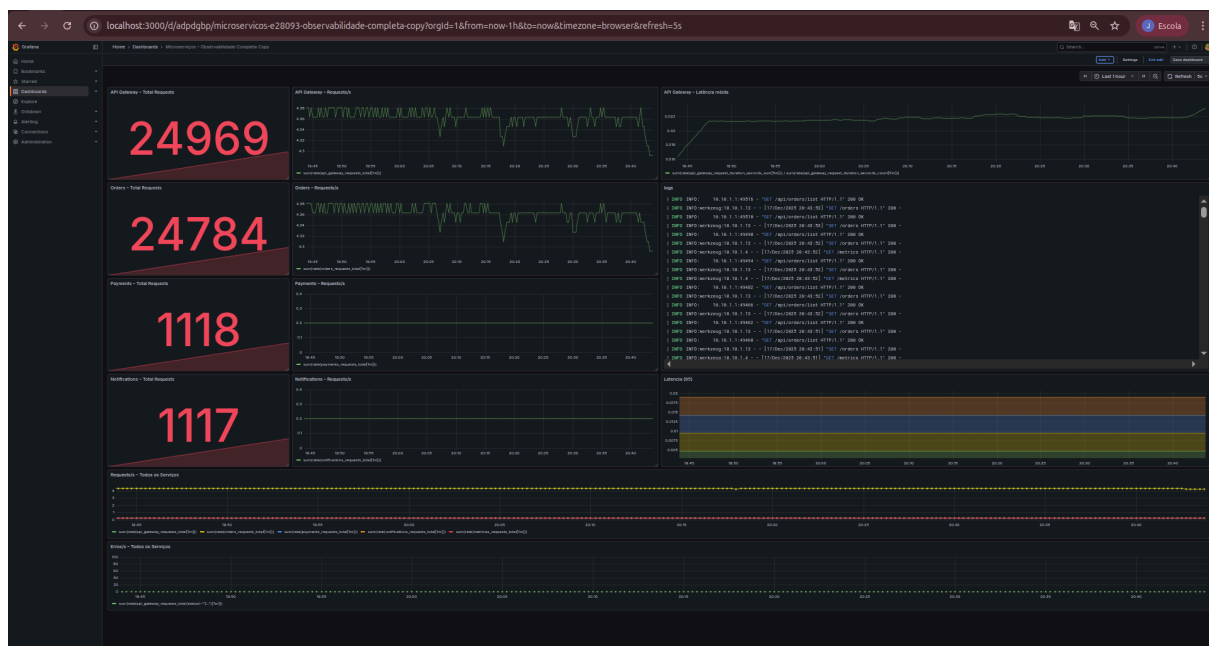
Visualização no Grafana

O **Grafana**, exposto na porta **3000**, consome os dados diretamente do Prometheus através de um **dataSource** configurado automaticamente via **provisioning**, eliminando a necessidade de configuração manual após o arranque do sistema.

Foram desenvolvidos **dashboards personalizados**, que permitem visualizar em tempo real:

- Total de pedidos por serviço;
- Requests por segundo (throughput);
- Latência média e percentil **p95**;
- Erros por segundo;
- Comparação de carga entre os diferentes serviços.

A **Figura abaixo** apresenta uma visão geral dos dashboards desenvolvidos, evidenciando métricas reais resultantes da geração de tráfego no sistema.



Os dashboards são carregados automaticamente no arranque do sistema, garantindo **persistência, reprodutibilidade e consistência** da visualização dos dados, mesmo após reinícios dos contentores.

5.2. Logs Centralizados (Loki e Promtail)

Para além das métricas, o sistema inclui uma solução completa de **centralização de logs**, permitindo analisar eventos e erros ocorridos nos serviços.

Recolha de Logs (Promtail)

O **Promtail** foi configurado para ler os logs diretamente dos contentores Docker, através do diretório:

```
/var/lib/docker/containers
```

Estes logs são recolhidos em tempo real e enviados para o **Loki**.

Armazenamento e Consulta (Loki)

O **Loki** atua como backend de logs, armazenando os registos e permitindo a sua consulta através de **LogQL**. Cada log mantém o contexto do contentor de origem, permitindo filtrar por serviço ou instância.

Visualização no Grafana

No Grafana, os logs podem ser explorados através do painel de logs, possibilitando:

- Filtragem por serviço
- Análise temporal de eventos
- Correlação entre métricas e logs

Esta integração permite uma abordagem de *observabilidade completa*, onde métricas e logs são analisados em conjunto para diagnóstico de problemas.

5.3. Considerações sobre Observabilidade

A integração de Prometheus, Grafana, Loki e Promtail fornece uma visão abrangente do sistema distribuído, permitindo:

- Monitorização contínua do desempenho
- Identificação de picos de carga
- Análise de latência e erros
- Centralização e consulta eficiente de logs

Esta abordagem segue boas práticas da indústria e demonstra a importância da observabilidade em arquiteturas baseadas em microserviços.

6. Interface e API Gateway

O acesso aos microsserviços é centralizado através de um **API Gateway**, responsável por expor uma interface única ao exterior e encaminhar os pedidos para os serviços internos adequados.

O ficheiro `main.py` do API Gateway foi adaptado para centralizar o acesso e expor uma interface unificada, definindo rotas públicas para `orders`, `payments` e `notifications`. O Gateway não contém lógica de negócio, funcionando apenas como ponto de entrada e observabilidade. As principais implementações neste componente incluem: * Encaminhamento dos pedidos para os serviços internos via HTTP. * Implementação de um endpoint de health check agregado (`/api/health`) que verifica o estado de todos os serviços.

6.1. API Gateway

Implementação do API Gateway

Inicialmente, foi considerada a utilização do **NGINX** como *Reverse Proxy*, de forma a ocultar a complexidade da arquitetura interna e encaminhar os pedidos HTTP para os diferentes microsserviços. No entanto, devido à complexidade adicional introduzida, bem como às limitações de tempo disponíveis, esta abordagem acabou por ser abandonada.

Em alternativa, foi implementado um **API Gateway desenvolvido em FastAPI**, que cumpre os seguintes objetivos:

- Centralização do acesso aos serviços
- Encaminhamento explícito de pedidos HTTP para cada microsserviço
- Instrumentação direta para métricas Prometheus
- Simplificação da observabilidade e do controlo de tráfego

O API Gateway expõe endpoints REST que reencaminham os pedidos para os serviços internos através da rede Docker, como por exemplo:

- `/api/orders/*` → Orders Service
- `/api/payments/*` → Payments Service
- `/api/notifications/*` → Notifications Service
- `/api/logs` → Logs Service
- `/api/metrics/summary` → Métricas Service

Além disso, o próprio API Gateway disponibiliza:

- Endpoint `/metrics` para recolha de métricas pelo Prometheus
- Endpoint `/api/health` para verificação do estado dos serviços dependentes

Esta abordagem permite manter uma separação clara entre clientes externos e os serviços internos, garantindo maior controlo e facilidade de manutenção.

6.2. Interface de Consumo da API

O projeto não inclui um frontend gráfico dedicado. Em vez disso, a interação com o sistema é realizada através de:

- Chamadas HTTP diretas (ex: `curl`)
- Dashboards do Grafana para visualização de métricas e logs
- Endpoints REST expostos pelo API Gateway

Esta decisão permitiu focar o desenvolvimento nos aspectos centrais do sistema distribuído, nomeadamente:

- Comunicação entre microsserviços
- Observabilidade
- Instrumentação correta de métricas
- Centralização de logs

A geração de tráfego para efeitos de teste e validação é realizada através de pedidos repetidos aos endpoints do API Gateway, o que permite observar, em tempo real, a atualização das métricas no Prometheus e a sua visualização no Grafana.

6.3. Considerações Finais sobre o Gateway

A implementação do API Gateway em FastAPI revelou-se uma solução eficaz e flexível, permitindo:

- Instrumentação direta com Prometheus
- Encaminhamento explícito e controlado de pedidos
- Simplificação da arquitetura face a soluções baseadas em proxies genéricos

Apesar de a utilização do NGINX ter sido inicialmente considerada, a solução final adotada mostrou-se mais adequada ao contexto académico e aos objetivos do pro

Decisões Técnicas e Limitações do Projeto

Tentativa de Implementação do API Gateway com NGINX

Durante o desenvolvimento do projeto, foi considerada a utilização do **NGINX** como *Reverse Proxy* para desempenhar o papel de **API Gateway**, com o objetivo de ocultar a topologia interna do sistema, centralizar o acesso aos microsserviços e encaminhar os pedidos HTTP para os serviços de backend com base em regras de roteamento por caminho (por exemplo, `/api/orders`, `/api/payments`, `/api/notifications`).

Esta abordagem permitiria uma separação clara entre o frontend e os serviços internos, aproximando a arquitetura de um cenário de produção real. No entanto, ao avançar com a implementação, verificou-se que a integração do NGINX introduzia uma complexidade significativamente superior à inicialmente prevista, sobretudo quando combinada com os restantes requisitos do sistema.

Em particular, surgiram dificuldades relacionadas com:

- A instrumentação e recolha de **métricas personalizadas** para o Prometheus;
- A correta exposição e encaminhamento do endpoint `/metrics` de todos os serviços;
- A **centralização e correlação de logs** através do Loki e Promtail;
- A necessidade de encaminhar simultaneamente tráfego de negócio e tráfego de observabilidade sem perda de informação ou inconsistências.

A configuração necessária para garantir que o NGINX não interferia com a recolha de métricas e logs exigia regras adicionais complexas, incluindo gestão de headers HTTP, múltiplos *upstreams*, reescrita de caminhos e tratamento específico de endpoints de observabilidade. Dada a limitação de tempo disponível para a conclusão do projeto e o foco principal nos conceitos de microsserviços e monitorização, esta abordagem revelou-se pouco viável no contexto académico.

Face a estas dificuldades, foi tomada a decisão técnica de **abandonar a utilização do NGINX como API Gateway** e implementar esta camada diretamente através de um serviço dedicado em **FastAPI**. Esta solução permitiu:

- Maior controlo sobre o encaminhamento de pedidos entre serviços;
- Instrumentação direta e consistente das métricas do gateway;
- Integração mais simples com Prometheus e Grafana;
- Redução da complexidade operacional e do risco de erros de configuração.

Esta decisão não invalida a relevância do NGINX em ambientes de produção mais avançados, mas reflete uma escolha pragmática, alinhada com os objetivos do projeto e com as restrições temporais existentes.

7. Conclusão

O desenvolvimento do presente projeto permitiu atingir integralmente os objetivos propostos no enunciado do Trabalho Prático, resultando na implementação bem-sucedida de um **sistema distribuído baseado em microsserviços**, totalmente containerizado e com uma stack de **observabilidade completa**. A evolução do projeto, desde serviços isolados e estáticos até uma arquitetura integrada e monitorizada, evidenciou a complexidade e as vantagens do paradigma de microsserviços.

Destacam-se as seguintes **conquistas técnicas**:

Desacoplamento de Serviços

A arquitetura baseada em microsserviços permitiu separar claramente as responsabilidades de cada componente (Orders, Payments, Notifications, Logs e Métricas). Esta separação favoreceu a escalabilidade, a manutenção independente dos serviços e a compreensão do fluxo global do sistema.

Persistência e Integridade de Dados

Apesar de alguns serviços utilizarem dados simulados, a arquitetura foi desenhada para suportar persistência real e consistência de informação, refletindo práticas comuns em sistemas distribuídos. A separação lógica dos serviços permite a fácil integração futura com bases de dados persistentes, mantendo a integridade dos dados e a rastreabilidade das operações.

Observabilidade Completa (Prometheus, Grafana, Loki e Promtail)

Um dos principais resultados do projeto foi a implementação de uma **stack de observabilidade real e funcional**:

- **Prometheus** foi utilizado para a recolha de métricas de todos os serviços, incluindo número total de pedidos, taxa de pedidos por segundo e latências (p95).
- **Grafana** foi configurado com dashboards personalizados, permitindo a visualização em tempo real do estado e desempenho de cada microserviço.
- **Loki**, em conjunto com o **Promtail**, permitiu a centralização e consulta de logs estruturados provenientes de todos os contentores, facilitando a análise de eventos e a detecção de problemas.

Esta abordagem proporcionou uma visão clara e integrada da saúde do sistema, aproximando o projeto de cenários reais de produção.

Centralização de Acesso através de API Gateway

O acesso ao sistema foi centralizado através de um **API Gateway implementado em FastAPI**, responsável pelo encaminhamento de pedidos para os diferentes serviços de backend. Esta solução permitiu:

- Ocultar a topologia interna da rede de microserviços;
- Simplificar a interação do frontend com o sistema;
- Instrumentar métricas específicas do gateway, monitorizadas pelo Prometheus.

A tentativa inicial de utilização do NGINX como API Gateway foi abandonada devido à complexidade adicional introduzida na integração com a stack de observabilidade, optando-se por uma solução mais controlável e adequada ao contexto do projeto.

Guia Prático de Demonstração do Sistema

1. Arranque do sistema

Na raiz do projeto:

```
docker compose up -d
```

Confirmar que todos os serviços estão ativos:

```
docker ps
```

2. Verificação de estado (Health Check)

```
curl http://localhost:8000/api/health
```

Este endpoint mostra o estado dos seguintes serviços:

- API Gateway
- Orders
- Payments
- Notifications
- Logs
- Métricas

3. Criação de uma order (fluxo automático)

```
curl -X POST http://localhost:8000/api/orders/create \  
-H "Content-Type: application/json" \  
-d '{"product":"Teclado","price":49.99}'
```

Resultado esperado:

- A order é criada
- Um payment é criado automaticamente
- Uma notification é criada automaticamente

4. Criação de múltiplas orders com valores diferentes

```
for i in {1..10}  
do  
  price=$((RANDOM % 500 + 10))  
  curl -s -X POST http://localhost:8000/api/orders/create \  
  -H "Content-Type: application/json" \  
  -d '{"product":"'Produto-$i'","price":$price}'  
done
```

Este comando gera tráfego real para testes de carga e observabilidade.

5. Listar orders

```
curl http://localhost:8000/api/orders/list
```

As orders apresentadas são lidas a partir da base de dados PostgreSQL.

6. Listar payments

```
curl http://localhost:8000/api/payments/list
```

Cada payment está associado a uma order através do campo order_id.

7. Listar notifications

```
curl http://localhost:8000/api/notifications/list
```

As notifications são criadas automaticamente após a criação das orders.

8. Acesso direto à base de dados PostgreSQL

Entrar no container do PostgreSQL:

```
docker exec -it postgres psql -U postgres -d sysd
```

Ver tabelas existentes:

```
\dt
```

Esperado:

- orders
- payments

Ver dados da tabela orders:

```
SELECT id, product, price, timestamp  
  
FROM orders  
  
ORDER BY id;
```

Ver dados da tabela payments:

```
SELECT id, order_id, amount, status, timestamp  
  
FROM payments  
  
ORDER BY id;
```

Ver relação entre orders e payments:

```
SELECT  
  
  o.id AS order_id,  
  
  o.product,  
  
  o.price,  
  
  p.id AS payment_id,  
  
  p.amount,  
  
  p.status  
  
FROM orders o  
  
JOIN payments p ON p.order_id = o.id  
  
ORDER BY o.id;
```

Sair da base de dados:

```
\q
```

9. Geração de erros para observabilidade

Endpoint inválido:

```
curl http://localhost:8000/api/orders/invalid
```

Geração de múltiplos erros:

```
for i in {1..10}; do  
    curl -s http://localhost:8000/api/invalid  
done
```

Os erros gerados ficam visíveis nos dashboards do Grafana.

10. Prometheus

Interface web:

```
http://localhost:9090
```

Verificação dos targets:

```
http://localhost:9090/targets
```

Todos os serviços devem aparecer com estado UP.

11. Grafana

Interface web:

```
http://localhost:3000
```

Credenciais:

- Utilizador: admin
- Palavra-passe: admin

Dashboards a demonstrar:

- Requests por segundo
- Latência média e p95
- Erros por segundo
- Comparação entre serviços

12. Logs (Loki e Promtail)

No Grafana:

- Abrir a secção Explore
- Selecionar o datasource Loki

Query exemplo:

```
{container_name="projeto-final-sysd_orders_1"}
```

13. Encerramento do sistema

docker compose down